



ПРОГРАММИРОВАНИЕ НА PYTHON

Модуль 2: Продвинутый уровень

Строки • Словари • Файлы • Модули • ООП

Глава 8: Строки и работа с текстом

8.1 Строки подробнее

Строка — это последовательность символов. Её можно нарезать, искать в ней, изменять регистр и многое другое.

```
text = "Привет, Python!"

print(len(text))      # длина строки: 15
print(text[0])        # первый символ: П
print(text[-1])       # последний: !
print(text[0:6])      # срез: Привет
```

- ✓ 15
- ✓ П
- ✓ !
- ✓ Привет

8.2 Срезы строк

Срез строка [начало:конец] возвращает часть строки. Конечный индекс — не включается.

```
s = "Python"

print(s[0:3])         # Pyt
print(s[3:])          # hon  (до конца)
print(s[:3])          # Pyt  (с начала)
print(s[::-1])        # nohtyP (вся строка задом наперёд)
```

- ✓ Pyt
- ✓ hon
- ✓ Pyt
- ✓ nohtyP

8.3 Методы строк

Метод	Что делает
<code>.upper()</code>	Все буквы заглавные
<code>.lower()</code>	Все буквы строчные
<code>.strip()</code>	Убирает пробелы по краям
<code>.replace(a, b)</code>	Заменяет a на b
<code>.split(",")</code>	Разбивает по символу → список
<code>.join(список)</code>	Склеивает список в строку
<code>.find(текст)</code>	Находит позицию подстроки
<code>.count(текст)</code>	Считает вхождения
<code>.startswith(текст)</code>	Начинается ли с...
<code>.isdigit()</code>	Состоит только из цифр?

```
name = "  Иван Петров  "
print(name.strip())          # "Иван Петров"
print(name.strip().upper())  # "ИВАН ПЕТРОВ"

phrase = "кот, пёс, рыба, птица"
animals = phrase.split(", ")
print(animals)               # ['кот', 'пёс', 'рыба', 'птица']

print(", ".join(animals))    # кот, пёс, рыба, птица
```

8.4 Форматирование строк (f-строки)

f-строки — самый удобный способ подставить переменные в текст. Ставь `f` перед кавычкой, а переменные оборачивай в `{}`.

```
name = "Катя"
age = 14
height = 165.5

print(f"Меня зовут {name}, мне {age} лет.")
print(f"Мой рост: {height:.1f} см")    # 1 знак после запятой
print(f"Через 10 лет мне будет {age + 10}")
```

- ✓ Меня зовут Катя, мне 14 лет.
- ✓ Мой рост: 165.5 см
- ✓ Через 10 лет мне будет 24

 **Задание 1:** Попроси пользователя ввести имя и год рождения. Выведи: «Привет, [имя]! В 2030 году тебе будет [X] лет.» Используй f-строку.

Глава 9: Словари (dict)

9.1 Что такое словарь?

Словарь — это коллекция пар «ключ: значение». Как настоящий словарь: ищешь слово (ключ) — находишь его перевод (значение).

```
student = {
    "name": "Артём",
    "age": 15,
    "grade": "9A"
}

print(student["name"])    # Артём
print(student["age"])     # 15
print(student.get("city", "не указан")) # не указан
```

□ **get() vs []**: Обращение через [] вызывает ошибку если ключа нет. Метод .get(ключ, значение_по_умолчанию) безопаснее.

9.2 Изменение словаря

```
person = {"name": "Лена", "age": 13}

person["age"] = 14          # изменить значение
person["city"] = "Москва"   # добавить новый ключ
del person["city"]          # удалить ключ

print(person) # {'name': 'Лена', 'age': 14}
```

9.3 Перебор словаря

```
scores = {"Алёша": 95, "Маша": 87, "Вова": 72}

# Перебор ключей
for name in scores:
    print(name)

# Перебор пар ключ-значение
for name, score in scores.items():
    print(f"{name}: {score} баллов")

# Только значения
for score in scores.values():
    print(score)
```

- ✓ Алёша: 95 баллов
- ✓ Маша: 87 баллов
- ✓ Вова: 72 баллов

9.4 Вложенные словари

```
school = {  
    "9А": {"teacher": "Иванова", "students": 28},  
    "9Б": {"teacher": "Петров", "students": 25},  
}  
  
print(school["9А"]["teacher"])    # Иванова  
print(school["9Б"]["students"])   # 25
```

 **Задание 2:** Создай словарь с информацией о себе: имя, возраст, город, любимый предмет. Выведи каждую пару в формате «ключ → значение».

Глава 10: Кортежи и множества

10.1 Кортеж (tuple)

Кортеж похож на список, но его нельзя изменить после создания. Используется для данных, которые не должны меняться.

```
coords = (55.75, 37.62) # координаты Москвы
colors = ("красный", "зелёный", "синий")

print(coords[0]) # 55.75
print(len(colors)) # 3

# Кортеж нельзя изменить:
# colors[0] = "жёлтый" -- ошибка!

# Удобная распаковка:
x, y = coords
print(f"Широта: {x}, Долгота: {y}")
```

□ **Когда использовать:** Список — когда данные могут меняться. Кортеж — когда данные постоянны (координаты, дата рождения, RGB-цвет).

10.2 Множество (set)

Множество хранит только уникальные элементы и не имеет порядка. Отлично подходит для удаления дубликатов.

```
# Удаление дубликатов:
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)
print(unique) # {1, 2, 3, 4}

# Операции с множествами:
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a & b) # пересечение: {3, 4}
print(a | b) # объединение: {1, 2, 3, 4, 5, 6}
print(a - b) # разность: {1, 2}
```

```
✓ {1, 2, 3, 4}
✓ {3, 4}
✓ {1, 2, 3, 4, 5, 6}
✓ {1, 2}
```

 **Задание 3:** Попроси пользователя ввести 5 слов (по одному). Сохрани их в список, потом преобразуй в множество и выведи — сколько уникальных слов было введено.

Глава 11: Работа с файлами

11.1 Запись в файл

Используй `open(имя, режим)` для работы с файлами. Конструкция `with` автоматически закрывает файл.

Режим	Описание
"w"	Запись (создаёт или перезаписывает файл)
"a"	Дозапись (добавляет в конец)
"r"	Чтение (файл должен существовать)
"r+"	Чтение и запись

```
# Запись
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Первая строка\n")
    f.write("Вторая строка\n")

# Дозапись
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Третья строка\n")
```

 **Важно:** Всегда указывай `encoding="utf-8"` при работе с русским текстом, иначе возможны ошибки.

11.2 Чтение из файла

```
# Прочитать весь файл
with open("notes.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# Читать построчно
with open("notes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip()) # strip() убирает \n в конце
```

11.3 Работа со списком строк

```
# Записать список
fruits = ["яблоко", "банан", "апельсин"]

with open("fruits.txt", "w", encoding="utf-8") as f:
    f.writelines(fruit + "\n" for fruit in fruits)

# Прочитать в список
with open("fruits.txt", "r", encoding="utf-8") as f:
    loaded = [line.strip() for line in f]

print(loaded) # ['яблоко', 'банан', 'апельсин']
```

 **Задание 4:** Напиши программу-дневник: пользователь вводит текст, программа добавляет его в файл `diary.txt` с новой строки. При запуске выводи все старые записи.

Глава 12: Модули и библиотеки

12.1 Что такое модуль?

Модуль — это готовый набор функций. Подключается командой `import`. Python поставляется с огромным количеством встроенных модулей.

12.2 Модуль `math`

```
import math

print(math.pi)           # 3.14159...
print(math.sqrt(16))     # 4.0   (квадратный корень)
print(math.pow(2, 10))   # 1024.0
print(math.ceil(3.2))    # 4     (округление вверх)
print(math.floor(3.9))   # 3     (округление вниз)
print(math.factorial(5)) # 120
```

12.3 Модуль `random`

```
import random

print(random.randint(1, 10))      # случайное целое 1-10
print(random.random())            # случайное 0.0 - 1.0
print(random.choice(["a", "б", "в"])) # случайный элемент

my_list = [1, 2, 3, 4, 5]
random.shuffle(my_list)           # перемешать список
print(my_list)
```

12.4 Модуль `datetime`

```
from datetime import datetime, date

now = datetime.now()
print(now)                # 2025-03-15 14:32:05
print(now.year)           # 2025
print(now.strftime("%d.%m.%Y")) # 15.03.2025

birthday = date(2010, 5, 20)
today = date.today()
age_days = (today - birthday).days
print(f"Тебе уже {age_days} дней!")
```

12.5 Установка внешних библиотек

Сотни тысяч дополнительных библиотек можно установить через `pip` в терминале:

```
# В терминале (не в Python-коде!):
pip install requests  # для работы с интернетом
pip install pygame    # для создания игр
pip install pillow     # для работы с картинками
```

 **Задание 5:** Создай программу «Лотерея»: с помощью `random` выбери 6 уникальных чисел от 1 до 49 (лотерея «Спортлото»). Используй `random.sample()`.

Глава 13: Обработка ошибок

13.1 Что такое исключение?

Когда программа сталкивается с ошибкой, Python «выбрасывает» исключение и останавливается. Но мы можем перехватить ошибку и обработать её!

```
# Без обработки — программа падает:
number = int(input("Введи число: ")) # если ввести "abc" — ОШИБКА!

# С обработкой — программа продолжает работу:
try:
    number = int(input("Введи число: "))
    print(f"Ты ввёл: {number}")
except ValueError:
    print("Ошибка! Это не число.")
finally:
    print("Программа завершена.") # выполняется всегда
```

□ **Блоки try/except/finally:** try — код, который может вызвать ошибку. except — что делать при ошибке. finally — выполняется в любом случае.

13.2 Виды ошибок

Тип ошибки	Когда возникает
<code>ValueError</code>	Неверный тип данных (<code>int('abc')</code>)
<code>ZeroDivisionError</code>	Деление на ноль (<code>5 / 0</code>)
<code>IndexError</code>	Выход за границы списка (<code>list[100]</code>)
<code>KeyError</code>	Ключ не найден в словаре
<code>FileNotFoundError</code>	Файл не существует
<code>TypeError</code>	Несовместимые типы (<code>'3' + 3</code>)

13.3 Несколько блоков except

```
try:
    x = int(input("Число: "))
    result = 100 / x
    print(f"100 / {x} = {result}")
except ValueError:
    print("Введи целое число!")
except ZeroDivisionError:
    print("На ноль делить нельзя!")
except Exception as e:
    print(f"Неожиданная ошибка: {e}")
```

 **Задание 6:** Напиши функцию `safe_divide(a, b)`, которая делит `a` на `b`. Если `b=0` — вернуть `None` и вывести предупреждение. Используй try/except.

Глава 14: Объектно-ориентированное программирование

14.1 Классы и объекты

ООП — это способ организации кода с помощью классов. Класс — это как шаблон (чертёж), а объект — конкретная вещь, созданная по этому шаблону.

□ **Аналогия:** Класс = архитектурный план дома. Объект = конкретный дом, построенный по этому плану. Можно построить много домов по одному плану.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} говорит: Гав!")

    def info(self):
        print(f"Имя: {self.name}, порода: {self.breed}")

# Создаём объекты:
dog1 = Dog("Шарик", "Дворняга")
dog2 = Dog("Рекс", "Немецкая овчарка")

dog1.bark()      # Шарик говорит: Гав!
dog2.info()      # Имя: Рекс, порода: Немецкая овчарка
```

- ✓ Шарик говорит: Гав!
- ✓ Имя: Рекс, порода: Немецкая овчарка

14.2 Атрибуты и методы

Понятие	Описание
<code>class</code>	Ключевое слово для создания класса
<code>__init__</code>	Конструктор: вызывается при создании объекта
<code>self</code>	Ссылка на сам объект (обязательный первый параметр)
<code>self.имя</code>	Атрибут — переменная объекта
<code>def метод(self)</code>	Метод — функция внутри класса

14.3 Более сложный пример

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Пополнение: {amount} руб. Баланс: {self.balance}")
```

```

def withdraw(self, amount):
    if amount > self.balance:
        print("Недостаточно средств!")
    else:
        self.balance -= amount
        print(f"Снятие: {amount} руб. Баланс: {self.balance}")

def __str__(self):
    return f"Счёт {self.owner}: {self.balance} руб."

# Использование:
acc = BankAccount("Алексей", 1000)
acc.deposit(500)
acc.withdraw(200)
acc.withdraw(2000)
print(acc)

```

- ✓ Пополнение: 500 руб. Баланс: 1500
- ✓ Снятие: 200 руб. Баланс: 1300
- ✓ Недостаточно средств!
- ✓ Счёт Алексей: 1300 руб.

14.4 Наследование

Класс может наследовать свойства другого класса — и добавлять собственные.

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("...")

class Cat(Animal): # Cat наследует Animal
    def speak(self):
        print(f"{self.name} говорит: Мяу!")

class Cow(Animal):
    def speak(self):
        print(f"{self.name} говорит: Муу!")

animals = [Cat("Мурка"), Cow("Бурёнка"), Cat("Барсик")]
for animal in animals:
    animal.speak()

```

- ✓ Мурка говорит: Мяу!
- ✓ Бурёнка говорит: Муу!
- ✓ Барсик говорит: Мяу!

 **Задание 7:** Создай класс Student с атрибутами name, grade и список grades (оценок). Добавь методы add_grade(оценка) и average() — возвращает средний балл. Создай 2-3 объекта и протестируй.

□ Итоговый проект: Записная книжка

Объединим всё изученное в реальном приложении — консольной записной книжке с сохранением данных в файл.

```
import json
import os

class Contact:
    def __init__(self, name, phone, email=""):
        self.name = name
        self.phone = phone
        self.email = email

    def to_dict(self):
        return {"name": self.name, "phone": self.phone, "email":
self.email}

    def __str__(self):
        return f"{self.name} | {self.phone} | {self.email}"

class PhoneBook:
    FILE = "contacts.json"

    def __init__(self):
        self.contacts = self.load()

    def load(self):
        if os.path.exists(self.FILE):
            with open(self.FILE, "r", encoding="utf-8") as f:
                data = json.load(f)
                return [Contact(**d) for d in data]
        return []

    def save(self):
        with open(self.FILE, "w", encoding="utf-8") as f:
            json.dump([c.to_dict() for c in self.contacts], f,
                ensure_ascii=False, indent=2)

    def add(self, name, phone, email=""):
        self.contacts.append(Contact(name, phone, email))
        self.save()
        print(f"Контакт {name} добавлен!")

    def search(self, query):
        results = [c for c in self.contacts
            if query.lower() in c.name.lower()]
        return results

    def show_all(self):
        if not self.contacts:
            print("Книжка пуста.")
        for i, c in enumerate(self.contacts, 1):
            print(f"{i}. {c}")

# Главное меню
def main():
    book = PhoneBook()
```

```
while True:
    print("\n1-Показать все  2-Добавить  3-Найти  4-Выход")
    choice = input("Выбор: ")
    if choice == "1":
        book.show_all()
    elif choice == "2":
        n = input("Имя: ")
        p = input("Телефон: ")
        e = input("Email (Enter=пропустить): ")
        book.add(n, p, e)
    elif choice == "3":
        q = input("Поиск: ")
        for c in book.search(q):
            print(c)
    elif choice == "4":
        break

main()
```

□ **Новое в проекте:** `import json` — модуль для хранения данных в формате JSON (удобная замена простым текстовым файлам). `os.path.exists()` — проверяет, существует ли файл.

 **Задание 8:** Улучши записную книжку: добавь возможность удалять контакт по номеру из списка. Используй метод `pop()` для списка.

□ Шпаргалка — Часть 2

```
# === СТРОКИ ===
s = "Hello, World!"
s.upper()          # "HELLO, WORLD!"
s.lower()          # "hello, world!"
s.replace("o", "0") # "Hell0, W0rld!"
s.split(", ")      # ["Hello", "World!"]
s[0:5]             # "Hello"
f"Имя: {name}"     # f-строка

# === СЛОВАРЬ ===
d = {"key": "value", "age": 20}
d["age"]           # 20
d.get("city", "-") # "-" если нет ключа
for k, v in d.items(): ...

# === КОРТЕЖ И МНОЖЕСТВО ===
t = (1, 2, 3)      # неизменяемый
s = {1, 2, 3}      # уникальные элементы
a & b               # пересечение
a | b              # объединение

# === ФАЙЛЫ ===
with open("f.txt", "w", encoding="utf-8") as f:
    f.write("текст")
with open("f.txt", "r", encoding="utf-8") as f:
    content = f.read()

# === TRY/EXCEPT ===
try:
    x = int(input())
except ValueError:
    print("Не число!")
finally:
    print("Готово")

# === КЛАСС ===
class MyClass:
    def __init__(self, x):
        self.x = x
    def show(self):
        print(self.x)

obj = MyClass(42)
obj.show()          # 42

# === МОДУЛИ ===
import math, random, datetime
math.sqrt(16)       # 4.0
random.randint(1, 6)
```

□ **Следующие шаги:** Pygame (игры) • Django/Flask (веб) • Pandas/Matplotlib (анализ данных) • NumPy (математика) • Tkinter (десктопные приложения) • Изучи алгоритмы на leetcode.com или codeforces.com